

**DEVELOPMENT OF LINE FOLLOWING GUIDED  
VEHICLE INTEGRATED WITH IOT**  
For  
**ELEMENTS OF MECHATRONICS SYSTEMS**  
**(21MHC101P)**

Submitted by

**VIJAY V (RA2311074010003)**  
**DEVENDU S DAS (RA2311074010005)**  
**MOHAN RAM P (RA2311074010006)**

**In Partial fulfilment for the degree**

of

**BACHELOR OF TECHNOLOGY**

in

**MECHATRONICS ENGINEERING**

of

**COLLEGE OF ENGINEERING & TECHNOLOGY**



S.R.M. Nagar, Kattankulathur, Chengalpet District

**May 2024**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**  
**(Under Section 3 of UGC Act, 1956)**

## BONAFIDE CERTIFICATE

Certified that this project report titled “*Line Following Guided Vehicle Integrated with Iot*” is the bonafide work of **VIJAY V (RA2311074010003), DEVENDU S DAS (RA2311074010005), and MOHAN RAM P (RA2311074010006)** who carried out the project work under our supervision. Certified further, project work reported here is for fulfilling the outcomes of the project based subject **ELEMENTS OF MECHATRONICS SYSTEMS (21MHC101P)**.

SIGNATURE

**Dr. R. RANJITH PILLAI, Ph.D.,**  
Faculty In charge,  
Department  
Dept. of Mechatronics Engineering

SIGNATURE

**Dr. G. MURALI, M.E., Ph.D.**  
Professor and Head of the  
Dept. of Mechatronics Engineering

Signature of the Internal Examiner1

Signature of the Internal Examiner 2

## **ABSTRACT**

Automated Guided Vehicles (AGVs) are an integral part of industries that transport goods without human intervention, ensuring product lines always have the materials they need without interruption.

The project focuses on the development of a connected line following guided vehicle integrated with IOT. It is capable of transferring materials from station to station based on the generated request.

The Automated Guided Line Following Vehicle was developed in the lab and it consists of key components like DC motor, motor drive, ESP32S, IR sensors and a joystick. 3D printed mounts were used to hold the IR sensors to the chassis.

The mobile robot consists of 4 IR sensors which help the robot to detect the line. An arena with an inventory and stations 1, 2, and 3 was developed for the navigation of the AVG. The robot is initially at the inventory and moves to the different stations according to the input given by the user. A joystick module was connected to a separate ESP and the reading on different inputs (Left, Right, Up, Down and Button) were observed. A communication was established between two ESPs (ESPNOW) - Interactive joystick to the robot. ESPNOW communication is integrated with the joystick system and the robot.

## **ACKNOWLEDGEMENTS**

It has been a great honor and privilege to undergo **B Tech** in **MECHTRONICS ENGINEERING** at **SRM Institute of Science and Technology**. We are very much thankful to the **Department of Mechatronics Engineering, SRM Institute of Science and Technology** for providing all facilities and support to meet our project requirements.

We would like to thank our Head of Department **Dr. G. MURALI, M.E., Ph.D.** and our teacher **Dr. R. RANJITH PILLAI, Ph.D.**, who gave us a big helping hand during the entire project.

We extend our gratitude to few of the Lab Assistants and Lab heads for helping us with information of few components used in our project.

The success and final outcome of this project required a lot of guidance and assistance from many people, and we are extremely fortunate that we have got this all from along the completion of our project work. Whatever we have done is only due to such guidance and assistance and we would like to thank them for their kind support.

**VIJAY V  
DEVENDU S DAS  
MOHAN RAM P**

# TABLE OF CONTENTS

| CHAPTER NO. | TITLE                                       | PAGE NO.  |
|-------------|---------------------------------------------|-----------|
|             | <b>ABSTRACT</b>                             | <b>3</b>  |
|             | <b>ACKNOWLEDGMENTS</b>                      | <b>4</b>  |
|             | <b>LIST OF TABLES</b>                       | <b>6</b>  |
|             | <b>LIST OF FIGURES</b>                      | <b>8</b>  |
|             | <b>LIST OF ABBREVIATIONS</b>                | <b>9</b>  |
|             | <b>LIST OF SYMBOLS</b>                      | <b>10</b> |
| <b>1</b>    | <b>INTRODUCTION AND PROPOSED WORK</b>       | <b>10</b> |
|             | 1.1 INTRODUCTION                            | 10        |
|             | 1.2 OBJECTIVES                              | 10        |
|             | 1.3 METHODOLOGY                             | 11        |
|             | 1.4 SCHEMATIC/BLOCK DIAGRAM                 | 12        |
| <b>2</b>    | <b>DESIGN AND FABRICATION</b>               | <b>13</b> |
|             | 2.1 CAD DESIGN                              | 13        |
|             | 3.1.1 FRAME                                 | 13        |
|             | 3.1.2 IR SENSOR HOLDER                      | 14        |
| <b>3</b>    | <b>ACTUATORS, SENSORS, AND INTERFACINGS</b> | <b>15</b> |
|             | 3.1 MOTORS                                  | 15        |
|             | 3.2 MOTOR DRIVER                            | 16        |
|             | 3.3 IR SENSOR                               | 17        |
|             | 3.4 ESP 32                                  | 17        |
|             | 3.5 JOYSTICK                                | 18        |
| <b>4</b>    | <b>RESULTS, DISCUSSION AND APPLICATIONS</b> | <b>19</b> |
|             | 4.1 WORKING                                 | 19        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| 4.2      | APPLICATIONS                       | 21        |
| <b>5</b> | <b>CONCLUSION AND FUTURE SCOPE</b> | <b>23</b> |
| 5.1      | CONCLUSION                         | 23        |
| 5.2      | FUTUTE SCOPE                       | 23        |
| <b>6</b> | <b>APPENDIX</b>                    | <b>24</b> |

## **LIST OF TABLES**

### **PAGE NO.**

|     |                     |    |
|-----|---------------------|----|
| 3.1 | MOTOR SPECIFICATION | 16 |
|-----|---------------------|----|

## **LIST OF FIGURES**

|                                                | <b>PAGE NO.</b> |
|------------------------------------------------|-----------------|
| 1.1. Block Diagram of Line Follower Robot      | 12              |
| 2.1 CAD Model of Line Following Guided Vehicle | 13              |
| 2.2 CAD Model of IR sensor holder              | 14              |
| 3.1 DC Motor                                   | 15              |
| 3.2 Motor Driver-L298N                         | 16              |
| 3.3 IR Sensor                                  | 17              |
| 3.4 ESP32                                      | 18              |
| 3.5 Joystick                                   | 18              |



## **ABBREVIATIONS**

CAD - Computer Aided Design

AGV-Automated Guided Vehicle

RPM - Revolutions Per Minute

# **CHAPTER-1**

## **INTRODUCTION**

Automated guided vehicle systems are used for tasks that would typically be handled by forklifts, conveyor systems or manual carts, moving large volumes of material in a repetitive manner. AGVs are used in a variety of applications. They're often used for transporting raw materials such as metal, plastic, rubber or paper. For example, AGVs can transport raw materials from receiving to the warehouse or deliver materials directly to production lines. AGVs consistently and reliably deliver raw materials needed without human intervention, ensuring that production lines always have the materials they need without interruption.

The automated guided vehicle (AGV) will be a major component of Industry 4.0. Nestled comfortably among physical machines, software and intelligence documents, advanced smart sensing, the global rise in ecommerce, etc., it helps to examine this one aspect in isolation.

The automated guided vehicle can be used to transport the product from station 1 to inventory through multiple assembly stations (station 2 and 3) as per guidance from the user. The product after manual inspection can be directed to its respective assembly stations (station 2 and 3) by the user by using joystick which wirelessly communicates with the AGV.

### **1.2 OBJECTIVES**

The main objective of the project is to develop a Line Following Guided vehicle that can be controlled wirelessly using a joystick to traverse between stations.

The sub objectives include:

- To develop a Line Follower robot with necessary electronics and sensors
- To develop joystick system to obtain the user input for the navigation
- To create an Arena necessary for the Line Follower navigation
- To Establish communication between two ESPs (ESP8266)- Interactive joystick to the robot
- Integrating and testing ESP8266 communication with the joystick system and the robot

## **1.3 METHODOLOGY**

### **1.3.1 Development of Line following bot**

The motor and wheels are assembled. The motor connections were made with esp32 through motor drive (l298n) and the PWM of both motors are adjusted. Four IR sensors were mounted and calibrated. The code for line following was developed and tested for simple straight line. Then certain mark for specific stations were made and the code was altered accordingly and was tested.

### **1.3.2 Development of Joystick system**

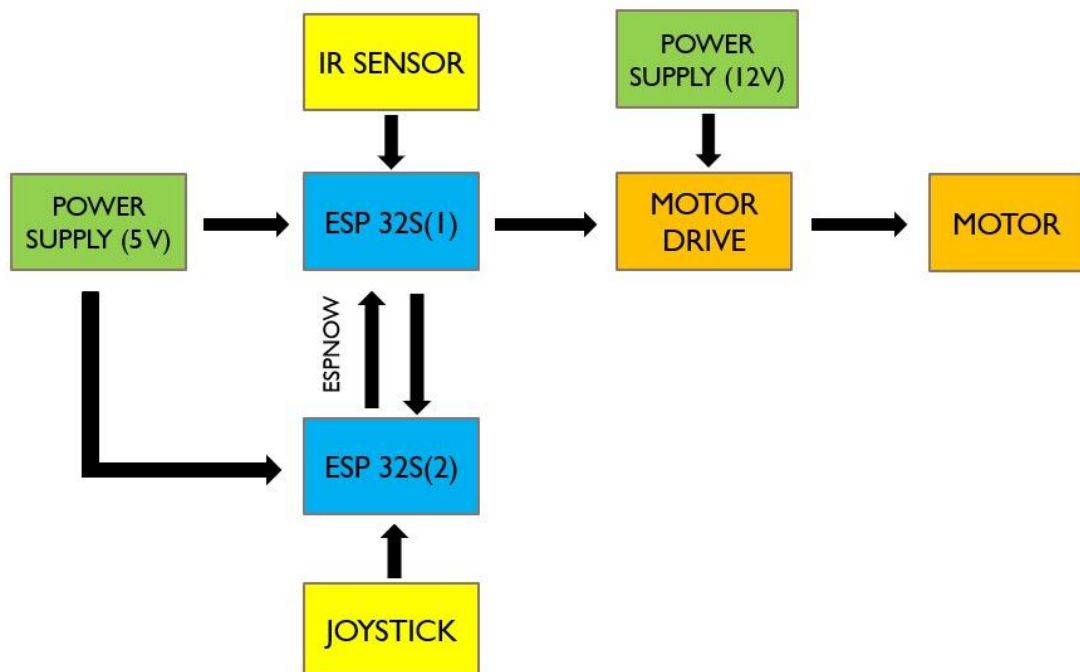
A joystick module was connected to a separate ESP and the reading on different input (Left, Right, Up, Down and Button) were observed.

### **1.3.3 Establishing ESP-ESP communication (ESPNow)**

Master (Transmitter) and slave (Receiver) example code for ESPNow protocol from esp32 library was uploaded into two ESPs and tested. A local communication between the two ESPs were successfully established and the slave was constantly receiving data which were being transmitted by the Master. Joystick was connected to the Master and changes were made in the code so that it sends data according to the input received from the joystick. The Line follower code was merged with slave code and edited in such a way that the line following algorithm works according to the data received from the master.

#### 1.4 BLOCK DIAGRAM

Figure 1.1 shows the complete electronic architecture of the project. In the AGV, the IR Sensors used for line detection are connected to an ESP32S(1) microcontroller. The motor drive connected to the ESP is used to control the motor and a 12 V battery is provided in order to supply power for the motor through the motor drive. The user inputs for navigation of the robot are given through a joystick which is connected to ESP32S(2). And in addition to this, a 5v power bank is used to supply power for both ESP(1) and ESP(2). Both the microcontrollers communicate with each other wirelessly by a communication protocol known as ESPNOW.



**Fig.1.1 Block diagram of the electronic system**

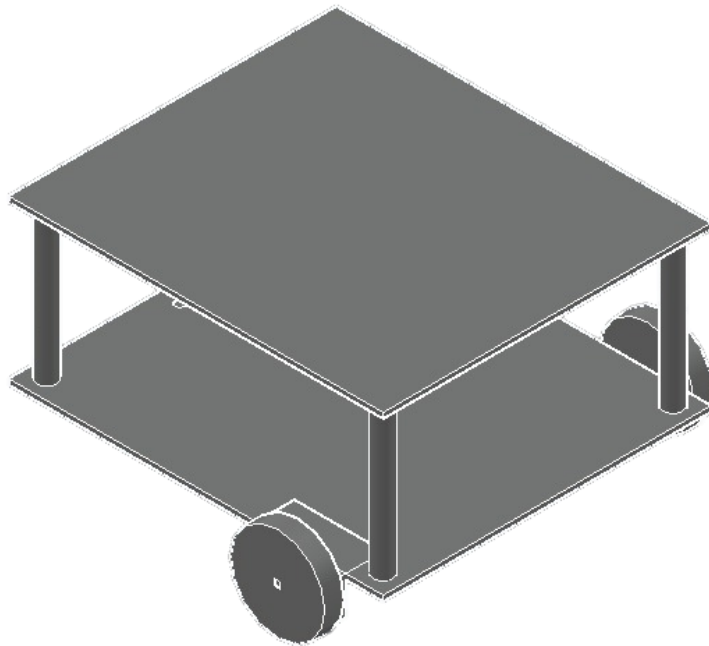
## **CHAPTER-2**

### **DESIGN AND FABRICATION**

#### **2.1 CAD DESIGN**

##### **2.1.1 Frame**

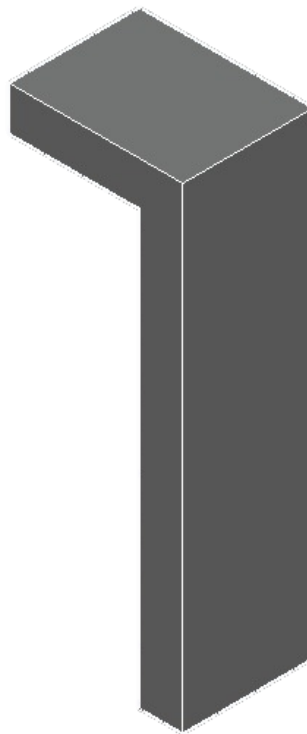
The figure 1.2 shows the model of the differential drive robot chassis which is designed using CAD software. In this, the IR sensor holder is attached in front of the chassis on which the IR sensor is mounted. The upper plate of the chassis is used to carry the product from inventory to the respective stations. All the electronic components including ESP32, motor driver, 12 V battery are placed on the lower plate of the chassis. The dimensions of the lower and upper plates are 40 x 35 x 0.5(cm). The height and diameter of cylindrical pillars are 16.5cm and 2cm respectively, and the radius of the wheel is 10cm.



**Fig.2.1 CHASIS**

### 2.1.2 IR Sensor Holder

The figure 1.3 shows CAD model of IR sensor holder which is attached in front of the chassis and with this the sensor is mounted on it. The holder makes sure that the sensor is stable in its position and avoids the wobble of the sensor so that it senses the path properly without making changes in the distance between the sensor and surface. The vertical portion has a dimension of 5cm and the horizontal portion has a dimension of 2cm.



**Fig.2.2 IR SENSOR HOLDER**

## CHAPTER 3

### ACTUATORS, SENSORS, AND INTERFACINGS

The electrical system of the Line Following Guided Vehicle consists of components such as the motor, motor drive, IR sensor, ESP32, joystick etc.

#### 3.1 MOTOR



**Fig.3.1 SPG30-30K DC motor**

The DC motor used here is SPG30-30K DC Geared Motor with Encoder 150RPM 18N.cm 12V. This Motor with Encoder features both advantages of position sensing as well as direction sensing of the rotating shaft. This application flexibility of the encoders will allow you to control the speed of the base motor very precisely. It is a compact brushed DC motor. This Geared Motor with Encoder is a low-cost solution for our robot Motor with Encoder equips 30:1 gearbox for giving the torque of 1.80 kg-cm. The output shaft of the motor is O-shaped which facilitates the better coupling. The shaft has the length of 15.5 mm with the diameter of 6mm. This is very useful especially for moving the robot to each station. The motor used is rated at 12V which will produce 150 RPM with the torque as high as 1.80 kg-cm. The motors are mounted at two sides of the chassis using a holder attached to it. Then the shaft of the motor is fixed to the wheels. The positive (M+) and negative (M-) terminals of the motor is connected to the output 4 and 5 of the motor drive.

**Table 3.1: Motor Specifications**

| Title           | Specification |
|-----------------|---------------|
| Rated voltage   | 12V           |
| Rated current   | 410mA         |
| Rated torque    | 1.80 KG-CM    |
| No-load current | 70mA          |
| Shaft Diameter  | 6mm           |

### 3.2 MOTOR DRIVE

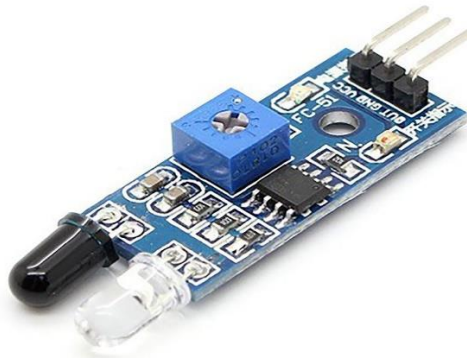
L298N is a motor driver which can control the speed and direction of up to two DC motors. This motor driver is one of the cheapest and the easiest way to control DC motors. There are two working concepts behind controlling the speed and the direction of a DC motor. Pulse width modulation is used to control the speed of the motor. H-bridge connections are used to change the direction of the motor. The speed of the DC motor depends on the input voltage it receives. So by varying the input voltage, the speed of a DC motor is varied. This process is called Pulse Width Modulation (PWM). Using the motor drive the PWM of both the motors are adjusted, setting them at an Analog value of 230 and 235 for the left and right motor respectively.

**Fig.3.2 Motor Drive-L298N**



### 3.3 IR SENSOR

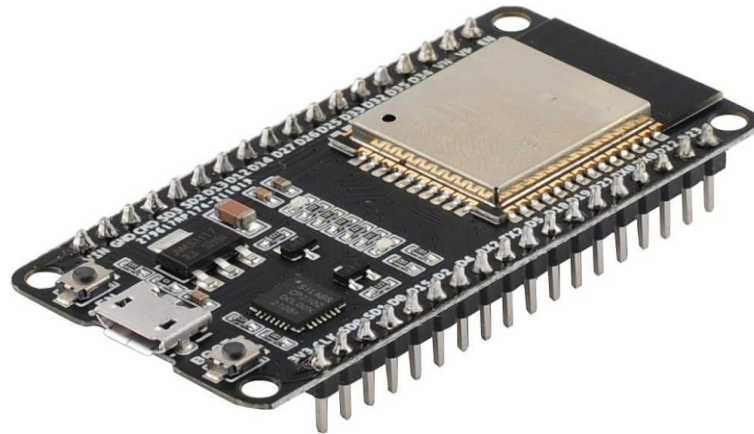
IR sensor is an electronic device that emits light in order to sense the object around the surroundings. An IR sensor is basically used to measure the heat of an object as well as detects the motion. Usually, in the infrared spectrum, all the objects radiate some form of thermal radiation. These types of radiations are invisible to our eyes, but infrared sensor can detect these radiations. The IR sensor used here is made to detect the black surface. The transmitter of the sensor sends the infrared signal, once which reflected back from the object received by the receiver. Since the infrared red rays observed by the black surface the receiver won't receive the infrared. This make the AVG to detect the black surface and make it move accordingly.



**Fig.3.3 IR Sensor**

### 3.4 ESP 32

ESP32 provides a series of low-cost, low-power system on a chip microcontrollers with integrated Wi-Fi and dual-mode Bluetooth. It is capable of functioning reliably in industrial environments, with an operating temperature ranging from  $-40^{\circ}\text{C}$  to  $+125^{\circ}\text{C}$ . This ESP32 is engineered for mobile devices, wearable electronics and IoT applications, since it achieves ultra-low power consumption with a combination of several types of proprietary software. It is highly-integrated with in-built antenna switches, RF balun, power amplifier, low-noise receive amplifier, filters, and power management modules. This enables with priceless functionality and versatility to the AGV with minimal Printed Circuit Board (PCB) requirements.



**Fig.3.4 ESP32**

### **3.5 JOYSTICK**

This is a dual-axis joystick designed for precision and durability. It provides two analog outputs, making it perfect for various applications. It is suitable for AGV projects. It Works well with ESP32 which offers flexibility in the projects. The quality constructions of this are made to last with a sturdy and long-lasting design. It provides a sense movement in 2 directions (axes). It also has an inbuilt switch which can be activated by pressing the stick. Directional movements are simply two potentiometers - one for each axis. Pots are ~10k each. The X and Y axes are two 10k potentiometers that control 2D movement by generating analog signals. This uses the 5V power supply. It is an input device that consists of a stick that can be tilted in multiple directions to control the device. The design of it is ergonomic and comfortable to use.



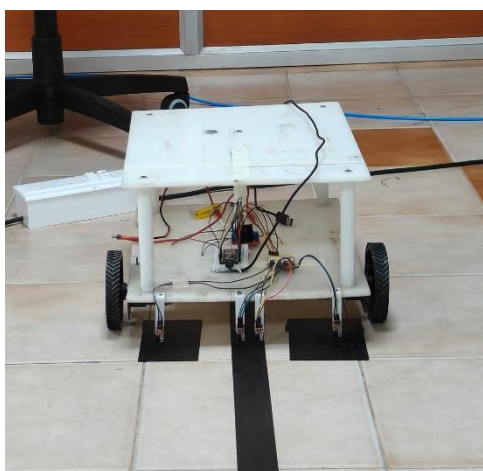
**Fig.3.5 Joystick**

## CHAPTER-4

### RESULTS, DISCUSSIONS AND APPLICATIONS

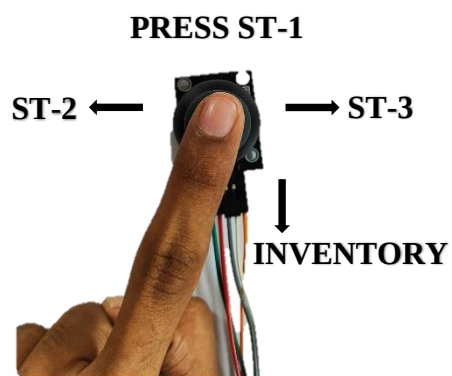
The Line Following AGV was developed and tested on the arena for all the following stations.

#### 4.1 WORKING



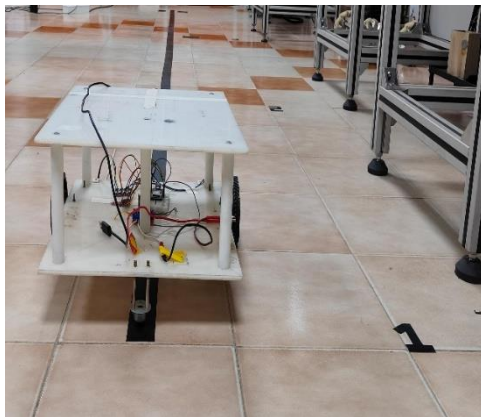
**Fig .4.1 Inventory**

Fig.4.1 shows the initial position the line following robot in the inventory from where the product is carried to station 1, 2 and 3.



**Fig.4.2 Joystick System**

Fig.4.2 shows the joystick system with the Station commands labelled. When the stick is pressed which indicates station-1. It tilted



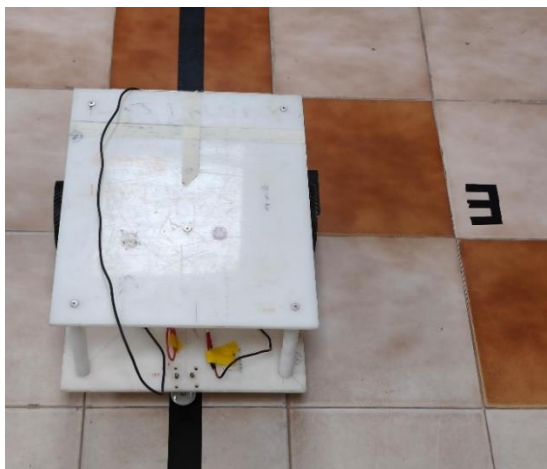
**Fig.4.3 Station 1**

In Fig. 4.3 the AGV is called to the station pressing the joystick, once the robot reaches on 1, it turns by an angle of 180 degrees and for the next command given by the user.



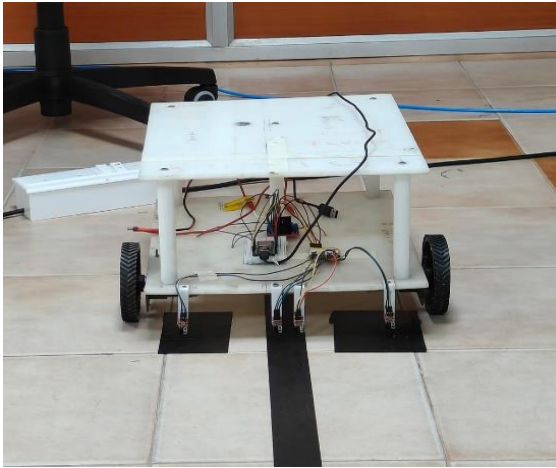
**Fig.4.4 Station 2**

Once the product is placed on the robot, the user can direct the robot to station 2 by tilting the joystick to the left side. As in Fig. 4.4, the AGV waits on station 2 till the next command.



**Fig.4.5 Station 3**

Fig. 4.4 shows that the robot is at station 3. When the user tilts the joystick to the right, the robot starts moving to station 3. It stops at the station 3 and waits for the next command.



**Fig4.6 Back At Inventory**

After assembly, the product is placed back onto the AGV. The bot is then sent back to inventory as the user turns the Joystick down. On reaching back at the inventory, it does a 180-degree turn.

## **4.2 APPLICATIONS**

- **Industrial Applications:** These robots can be used as automated equipment carriers in industries replacing traditional conveyer belts. Line-following robots are used in industrial settings for tasks such as material handling, assembly line operations, and warehouse management. They can transport goods along designated paths efficiently, reducing the need for human intervention.
- **Agriculture:** Line-following robots equipped with sensors can be used in agriculture for tasks like crop monitoring, pest control, and irrigation. They can navigate through fields to collect data or perform targeted actions based on crop health and environmental conditions.
- **Healthcare:** In hospitals and healthcare facilities, line-following robots can assist in delivering medication, transporting medical supplies, or navigating through the facility to provide assistance to patients and staff.
- **Security and Surveillance:** Line-following robots equipped with cameras and sensors can patrol designated areas for security purposes. They can detect anomalies, monitor activities, and provide real-time alerts in case of intrusions or emergencies.

- **Public Transportation:** Line-following robots can be deployed in public transportation systems, such as airports and train stations, to guide passengers to their destinations or assist in luggage handling.
- These are just a few examples of the diverse applications of line-following robots. As technology advances and new capabilities are developed, their potential uses are likely to expand even further.

## CHAPTER-5

### CONCLUSION AND FUTURE SCOPE

#### 5.1 CONCLUSION

The learning and outcomes of during the project session were precious and it provided a great opportunity to learn about many new things and it provided a bequeath experience to handle things and to build them in pre-eminent way out of many possible ways that exists in the current time.

This was a great period completing the first ever major project of our engineering life and had the huge opportunity to learn comprehend things from our Course instructor, assistants and friends from other batches. Without their guidance it wouldn't have been possible today.

The project is considered to be a stepping stone and serves as an interest to inspire us for the projects in the upcoming year.

#### 5.2 FUTURE SCOPE

1. **Multi-functional Capabilities:** Future AGVs may be designed to perform a wider range of tasks beyond material handling and transportation. They could be equipped with modular attachments or specialized tools to support tasks such as palletizing, assembly, inspection, and maintenance in manufacturing, warehousing, and logistics environments. This versatility would enhance their utility and cost-effectiveness across various industries.
2. **Collaborative Robotics:** AGVs will increasingly collaborate with other robots and human workers to optimize workflow efficiency and flexibility. Collaborative robots (cobots) working alongside AGVs will enable tasks such as cooperative picking, sorting, and packaging, enhancing productivity and adaptability in dynamic production environments.
3. **Adaptability to customized workflows:** AGVs will be designed to adapt seamlessly to customized workflows and production requirements, enabling rapid reconfiguration and scalability in response to changing demand and production schedules. This flexibility will enable businesses to optimize resource utilization and respond quickly to market dynamics.

## CHAPTER - 6

### APPENDIX

#### CODES:

##### MASTER

```
#include <esp_now.h>
#include <WiFi.h>
#include <esp_wifi.h> // only for esp_wifi_set_channel()
// Global copy of slave
esp_now_peer_info_t slave;
#define CHANNEL 1
#define PRINTSCANRESULTS 0
#define DELETEBEFOREPAIR 0
// Init ESP Now with fallback
void InitESPNow() {
  WiFi.disconnect();
  if (esp_now_init() == ESP_OK) {
    Serial.println("ESPNow Init Success");
  }
  else {
    Serial.println("ESPNow Init Failed");
    // Retry InitESPNow, add a counte and then restart?
    // InitESPNow();
    // or Simply Restart
    ESP.restart();
  }
}
// Scan for slaves in AP mode
void ScanForSlave() {
  int16_t scanResults = WiFi.scanNetworks(false, false, false, 300, CHANNEL); //
  Scan only on one channel
  // reset on each scan
```



```

bool slaveFound = 0;
memset(&slave, 0, sizeof(slave));
Serial.println("");
if (scanResults == 0) {
    Serial.println("No WiFi devices in AP Mode found");
} else {
    Serial.print("Found "); Serial.print(scanResults); Serial.println(" devices ");
    for (int i = 0; i < scanResults; ++i) {
        // Print SSID and RSSI for each device found
        String SSID = WiFi.SSID(i);
        int32_t RSSI = WiFi.RSSI(i);
        String BSSIDstr = WiFi.BSSIDstr(i);
        if (PRINTSCANRESULTS) {
            Serial.print(i + 1);
            Serial.print(": ");
            Serial.print(SSID);
            Serial.print(" ");
            Serial.print(RSSI);
            Serial.print(" ");
            Serial.println("");
        }
        delay(10);
        // Check if the current device starts with Slave
        if (SSID.indexOf("Slave") == 0) {
            // SSID of interest
            Serial.println("Found a Slave.");
            Serial.print(i + 1); Serial.print(": "); Serial.print(SSID); Serial.print(" ");
            Serial.print(BSSIDstr); Serial.print(" "); Serial.print(" "); Serial.print(RSSI);
            Serial.print(" "); Serial.println("");
            // Get BSSID => Mac Address of the Slave
            int mac[6];
            if ( 6 == sscanf(BSSIDstr.c_str(), "%x:%x:%x:%x:%x:%x", &mac[0], &mac[1],
            &mac[2], &mac[3], &mac[4], &mac[5]) ) {
                for (int ii = 0; ii < 6; ++ii ) {

```

```

        slave.peer_addr[ii] = (uint8_t) mac[ii];
    }
}

slave.channel = CHANNEL; // pick a channel
slave.encrypt = 0; // no encryption
slaveFound = 1;

// we are planning to have only one slave in this example;
// Hence, break after we find one, to be a bit efficient
break;
} } }

if (slaveFound) {
    Serial.println("Slave Found, processing..");
} else {
    Serial.println("Slave Not Found, trying again.");
}

// clean up ram
WiFi.scanDelete();
}

// Check if the slave is already paired with the master.
// If not, pair the slave with master
bool manageSlave() {
    if (slave.channel == CHANNEL) {
        if (DELETEBEFOREPAIR) {
            deletePeer();
        }
        Serial.print("Slave Status: ");
        // check if the peer exists
        bool exists = esp_now_is_peer_exist(slave.peer_addr);
        if (exists) {
            // Slave already paired.
            Serial.println("Already Paired");
            return true;
        } else {
            // Slave not paired, attempt pair

```

```

    esp_err_t addStatus = esp_now_add_peer(&slave);
    if (addStatus == ESP_OK) {
// Pair success
        Serial.println("Pair success");
        return true;
    } else if (addStatus == ESP_ERR_ESPNOW_NOT_INIT) {
        // How did we get so far!!
        Serial.println("ESPNow Not Init");
        return false;
    } else if (addStatus == ESP_ERR_ESPNOW_ARG) {
        Serial.println("Invalid Argument");
        return false;
    } else if (addStatus == ESP_ERR_ESPNOW_FULL) {
        Serial.println("Peer list full");
        return false;
    } else if (addStatus == ESP_ERR_ESPNOW_NO_MEM) {
        Serial.println("Out of memory");
        return false;
    } else if (addStatus == ESP_ERR_ESPNOW_EXIST) {
        Serial.println("Peer Exists");
        return true;
    } else {
        Serial.println("Not sure what happened");
        return false;
    }
}

} else {
    // No slave found to process
    Serial.println("No Slave found to process");
    return false;
}
}

void deletePeer() {
    esp_err_t delStatus = esp_now_del_peer(slave.peer_addr);
    Serial.print("Slave Delete Status: ");

```

```

if (delStatus == ESP_OK) {
    // Delete success
    Serial.println("Success");
} else if (delStatus == ESP_ERR_ESPNOW_NOT_INIT) {
    // How did we get so far!!
    Serial.println("ESPNow Not Init");
} else if (delStatus == ESP_ERR_ESPNOW_ARG) {
    Serial.println("Invalid Argument");
} else if (delStatus == ESP_ERR_ESPNOW_NOT_FOUND) {
    Serial.println("Peer not found.");
} else {
    Serial.println("Not sure what happened");
}
}

uint8_t data;
int b;
int a;
int c;

// send data
void sendData() {
    const uint8_t *peer_addr = slave.peer_addr;
    Serial.print("Sending: "); Serial.println(data);
    esp_err_t result = esp_now_send(peer_addr, &data, sizeof(data));
    Serial.print("Send Status: ");
    if (result == ESP_OK) {
        Serial.println("Success");
    } else if (result == ESP_ERR_ESPNOW_NOT_INIT) {
        // How did we get so far!!
        Serial.println("ESPNow not Init.");
    } else if (result == ESP_ERR_ESPNOW_ARG) {
        Serial.println("Invalid Argument");
    } else if (result == ESP_ERR_ESPNOW_INTERNAL) {
        Serial.println("Internal Error");
    } else if (result == ESP_ERR_ESPNOW_NO_MEM) {
        Serial.println("ESP_ERR_ESPNOW_NO_MEM");
    }
}

```

```

    } else if (result == ESP_ERR_ESPNOW_NOT_FOUND) {
        Serial.println("Peer not found.");
    } else {
        Serial.println("Not sure what happened");
    }
}

// callback when data is sent from Master to Slave
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
    char macStr[18];
    snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4],
             mac_addr[5]);
    Serial.print("Last Packet Sent to: "); Serial.println(macStr);
    Serial.print("Last Packet Send Status: "); Serial.println(status ==
ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery Fail");
    Serial.println(b);
}

int x=33;
int s=15;
int y=35;
void setup() {
    pinMode(x,INPUT);
    pinMode(s,INPUT);
    pinMode(y,INPUT);
    pinMode(s,INPUT_PULLUP);
    Serial.begin(115200);
    //Set device in STA mode to begin with
    WiFi.mode(WIFI_STA);
    esp_wifi_set_channel(CHANNEL, WIFI_SECOND_CHAN_NONE);
    Serial.println("ESPNow/Basic/Master Example");
    // This is the mac address of the Master in Station Mode
    Serial.print("STA MAC: "); Serial.println(WiFi.macAddress());
    Serial.print("STA CHANNEL "); Serial.println(WiFi.channel());
    // Init ESPNow with a fallback logic
    InitESPNow();
}

```

```

// Once ESPNow is successfully Init, we will register for Send CB to
// get the status of Trasnmitted packet
esp_now_register_send_cb(OnDataSent);
}

void loop() {
  // In the loop we scan for slave
  ScanForSlave();
  // If Slave is found, it would be populate in slave variable
  // We will check if slave is defined and then we proceed further
  if (slave.channel == CHANNEL) { // check if slave channel is defined
    // slave is defined
    // Add slave as peer if it has not been added already
    bool isPaired = manageSlave();
    if (isPaired) {
      a=analogRead(x);          // x axis
      b=digitalRead(s);         // button
      c=analogRead(y);          // y axis
      if(c==0){ // DOWN
        data=0;
        sendData();}
      else if(b==0){ //BUTTON
        data=1;
        sendData();}
      else if(a==0){ //LEFT
        data=2;
        sendData();}
      else if(a>4000){ //RIGHT
        data=3;
        sendData();
      }else{
        delay(10);
      }
    }
    // pair success or already paired
    // Send data to device
  }
}

```

```

    } else {
        // slave pair failed
        Serial.println("Slave pair failed!");
    } }
else {
    // No slave found to process
}
// wait for 3seconds to run the logic again
}

```

## **SLAVE**

```

#include <esp_now.h>
#include <WiFi.h>
#define CHANNEL 1
//MOTOR pins
int l1=14;
int l2=27;
int r1=15;
int r2=4;
//PWM pins
int enr=33;
int enl=32;
//IR pins
int ir1=22;
int ir2=17;
int ir3=3;
int ir4=21;
int a,b,c,d;
//STATIONS
void ST1(){          //STATION 1
do{
  lf();
}while(b==1||c==1);

```

```

}
void ST2(){          //STATION 2
do{
  lf();
}while(a==0);
}
void ST3(){          //STATION 3
do{
  lf();
}while(d==0);
}
//Line follower
void lf(){
  a=digitalRead(ir4);
  b=digitalRead(ir3);
  c=digitalRead(ir2);
  d=digitalRead(ir1);
  Serial.print(a);
  Serial.print(" _ ");
  Serial.print(b);
  Serial.print(" _ ");
  Serial.print(c);
  Serial.print(" _ ");
  Serial.println(d);
  if(b==1 && c==1){
    digitalWrite(r1,LOW);
    digitalWrite(r2,HIGH);  //FORWARD
    digitalWrite(l1,HIGH);
    digitalWrite(l2,LOW);
  }
  else if(b==0 && c==1){
    digitalWrite(l1,HIGH);
    digitalWrite(l2,LOW);  //TURNS RIGHT
    digitalWrite(r1,LOW);
  }
}

```



```

digitalWrite(r2,LOW);
}
else if(b==1 && c==0){
digitalWrite(r1,LOW);
digitalWrite(r2,HIGH); //TURNS LEFT
digitalWrite(l1,LOW);
digitalWrite(l2,LOW);
}
else if(b==0 && c==0){
digitalWrite(r1,LOW);
digitalWrite(r2,LOW); //DERAIL
digitalWrite(l1,LOW);
digitalWrite(l2,LOW);
}}
void rel(){
digitalWrite(r1,LOW);
digitalWrite(r2,LOW);
digitalWrite(l1,LOW);
digitalWrite(l2,LOW);
delay(1000);
}
void turn0(){
do{
a=digitalRead(ir4);
b=digitalRead(ir3);
c=digitalRead(ir2);
d=digitalRead(ir1);
digitalWrite(r1,LOW);
digitalWrite(r2,HIGH); //turn the bot at inventory
digitalWrite(l1,LOW);
digitalWrite(l2,HIGH);
}while(a==0 || b==0 || c==0 || d==0);
}
void turn1(){

```

```

do{
a=digitalRead(ir4);
b=digitalRead(ir3);
c=digitalRead(ir2);
d=digitalRead(ir1);
digitalWrite(r1,LOW);
digitalWrite(r2,HIGH); //turn the bot at station 1
digitalWrite(l1,LOW);
digitalWrite(l2,HIGH);
}while(b==0 || c==0);
}

void turn2(){
do{
a=digitalRead(ir4);
b=digitalRead(ir3);
c=digitalRead(ir2);
d=digitalRead(ir1);
digitalWrite(r1,LOW);
digitalWrite(r2,HIGH); //turn the bot at station 2
digitalWrite(l1,LOW);
digitalWrite(l2,HIGH);
}while(b==0 || c==0 || a==1);
}

void turn3(){
do{
a=digitalRead(ir4);
b=digitalRead(ir3);
c=digitalRead(ir2);
d=digitalRead(ir1);
digitalWrite(r1,LOW);
digitalWrite(r2,HIGH); //turn the bot at station 3
digitalWrite(l1,LOW);
digitalWrite(l2,HIGH);
}while(b==0 || c==0 || d==1);
}

```

```

}

//IR CHECK
int check(){          //checks the current position of the bot
a=digitalRead(ir4);
b=digitalRead(ir3);
c=digitalRead(ir2);
d=digitalRead(ir1);
if(a==1 && d==1){ return 0; }    //bot is at inventory
else if(a==0 && d==0){ return 1; } //bot is at station 1
else if(a==1 && d==0){ return 2;}  //bot is at station 2
else if(a==0 && d==1){ return 3;}  //bot is in station 3
}

// Init ESP Now with fallback
void InitESPNow() {
  WiFi.disconnect();
  if (esp_now_init() == ESP_OK) {
    Serial.println("ESPNow Init Success");
  }
  else {
    Serial.println("ESPNow Init Failed");
    // Retry InitESPNow, add a counte and then restart?
    // InitESPNow();
    // or Simply Restart
    ESP.restart();
  }
}

// config AP SSID
void configDeviceAP() {
  const char *SSID = "Slave_1";
  bool result = WiFi.softAP(SSID, "Slave_1_Password", CHANNEL, 0);
  if (!result) {
    Serial.println("AP Config failed.");
  } else {
    Serial.println("AP Config Success. Broadcasting with AP: " + String(SSID));
  }
}

```

```

    Serial.print("AP CHANNEL "); Serial.println(WiFi.channel());
  }}

void setup() {
  pinMode(l1,OUTPUT);
  pinMode(l2,OUTPUT);
  pinMode(r1,OUTPUT);
  pinMode(r2,OUTPUT);
  pinMode(enr,OUTPUT);
  pinMode(enl,OUTPUT);
  analogWrite(enr,230);
  analogWrite(enl,235);
  pinMode(ir1,INPUT);
  pinMode(ir2,INPUT);
  pinMode(ir3,INPUT);
  pinMode(ir4,INPUT);

  Serial.begin(115200);

  Serial.println("ESPNow/Basic/Slave Example");

  //Set device in AP mode to begin with
  WiFi.mode(WIFI_AP);

  // configure device AP mode
  configDeviceAP();

  // This is the mac address of the Slave in AP Mode
  Serial.print("AP MAC: "); Serial.println(WiFi.softAPmacAddress());

  // Init ESPNow with a fallback logic
  InitESPNow();

  // Once ESPNow is successfully Init, we will register for recv CB to
  // get recv packer info.
  esp_now_register_recv_cb(OnDataRecv);
}

// callback when data is recv from Master
void OnDataRecv(const uint8_t *mac_addr, const uint8_t *data, int data_len) {
  char macStr[18];
  snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",

```

```

        mac_addr[0],  mac_addr[1],  mac_addr[2],  mac_addr[3],  mac_addr[4],
mac_addr[5]);
    Serial.print("Last Packet Recv from: "); Serial.println(macStr);
    Serial.print("Last Packet Recv Data: "); Serial.println(*data);
    Serial.println("");
//DOWN
if(*data==0){
    if(check()==0){                // if bot is at inventory
        rel();
    }else if(check()==1){          // if bot is at station 1
        ST1();
        rel();
        turn0();
        rel();
    }else if(check()==2){          // if bot is at station 2
        ST1();
        rel();
        turn0();
        rel();
    }else if(check()==3){          // if bot is at station 3
        ST1();
        rel();
        turn0();
        rel();
    }
}
//BUTTON
if(*data==1){
    if(check()==0){                // if bot is at inventory
        ST1();
        rel();
        turn1();
        rel();
    }
    else if(check()==1){           // if bot is at station 1

```

```

    rel();
}

else if(check()==2){    // if bot is at station 2
    delay(1000);
    turn2();
    rel();
    ST1();
    rel();
    turn1();
    rel();
}
else if(check()==3){    // if bot is at station 3
    delay(1000);
    turn3();
    rel();
    ST1();
    rel();
    turn1();
    rel();
} }
//LEFT
else if(*data==2){
    if(check()==0){    // if bot is at inventory
        rel();
    }
    else if(check()==1){    // if bot is at station 1
        ST2();
        rel();}
    else if(check()==2){    // if bot is at station 2
        rel();}
    else if(check()==3){    // if bot is at station 3
        delay(1000);
        turn3();

```

```

    rel();
    ST3();
    rel();
    turn3();
    rel();
} }
// RIGHT
else if(*data==3){
    if(check()==0){        // if bot is at inventory
        rel();
    }
    if(check()==1){        // if bot is at station 1
        ST3();
        rel();}
    else if(check()==2){    // if bot is at station 3
        ST3();
        rel();
    }
    else if(check()==3){    // if bot is at station 3
        rel();
    } }
else{
    rel();}
}
void loop() {
    // Chill
}

```